RECOMMENDATION SYSTEMS FOR PRACTITIONERS

Production Recommendation Systems

An Engineering Guide to Collaborative Filtering, Content-Based Models, Hybrid Blends, Candidate Generation, Serving & MLOps, and Capstone Matrix Factorization Pipelines

Lamhot Siagian • **AI Engineering Insider**

Like a hen feeding her brood, a recommender serves each user just what is relevant: explicit & implicit feedback, cosine & content engines, hybrid blends, evaluation, sequential models, ANN candidate generation, capstone matrix factorization, and the serving & MLOps that run it --- one runnable project.

Production Recommendation Systems.

This book accompanies the open `recommendation-system-rag` reference project (FastAPI + LangGraph + pgvector backend, Next.js lab UI). Every code listing is drawn from or directly runnable against that project, and every chapter can be verified interactively through its matching lab in the web UI.

Code samples are provided for study and reference. Model names, library versions, and database structures change over time; verify against current documentation before production use.

Contact: aiengineeringinsider.substack.com

Contents

How to Use This Book

This is a *hands-on system-design* book. It is built around one reproducible project — the `recommendation-system-rag` application — and teaches production recommendation systems by dissecting that project module by module. Every chapter follows the same rhythm:

1. **Overview** — the concept, the trade-offs, and where it sits in the recommendation pipeline.
2. **Architecture diagram** — a TikZ view of the components and data flow for that chapter.
3. **Production code** — the real backend module that implements the concept, explained line by line.
4. **Test it in the UI** — the exact slash command to type in the chat interface or the progress modal, so the theory is verifiable, not abstract.
5. **Evaluation** — interview-grade questions (conceptual, debugging, and production) with full worked answers.

⋆ Key Insight

The fastest way to learn this material is to read a chapter, then immediately open the chat interface, trigger the slash commands, and review output details. Recommendation engines are full of mathematical feedback loops; seeing the numbers move in the UI builds intuition that prose cannot.

Running the reference project

Shell: bring up the project

```
# 1. Backend (FastAPI + LangGraph + pgvector)
ollama pull llama3.1 && ollama pull nomic-embed-text
docker compose up -d            # Postgres + pgvector
cd backend && pip install -r requirements.txt
uvicorn app.main:app --reload   # http://localhost:8000/api/v1

# 2. Frontend lab UI (Next.js)
cd frontend && npm install && npm run dev   # http://localhost:3000
```

Chapter map

Table 1 lists every chapter, its core focus, the backend module it dissects, and the interactive command that lets you test it live.

#	Chapter focus	Backend module dissected	Slash Command
1	Explicit vs Implicit Feedback	`course/commands.py` (sentiment classification)	`/classify-feedback`
2	Collaborative Filtering Similarity Engines	`course/commands.py` (Cosine user matching)	`/tenant-similar-users`
3	Content-Based Filtering	`course/commands.py` (Vector memory profiles)	`/memory-user-profile`
4	Hybrid Systems	`course/commands.py` (Collab + Memory blending)	`/hybrid-mix-full`
5	Knowledge-Based	`course/commands.py` (Cold start vs Warm start)	`/warm-start-sim`
6	Recommender Evaluation	`course/commands.py` (RMSE & tenant benchmarks)	`/tenant-evaluate`
7	Neural & Sequential Models	`course/commands.py` (Markov transition sequence)	`/memory-sequence-train`
8	Candidate Gen & ANN	`course/commands.py` (Brute-force vs ANN index scan)	`/tenant-scoped-ann`
9	Capstone Production pipeline	`course/commands.py` (SVD + memory boost)	`/capstone-recommend`

Table 1: Chapter map: each chapter dissects one module of the course project and is verified through a matching interactive command in the UI.

About this edition

This build includes the full scaffold, all nine chapters (1–9) representing the full recommendation systems curriculum, each grounded in the real backend and verifiable through its matching commands in the UI.

PART I

Foundations & Filtering

Before building complex recommenders, you must understand what signal types you are working with, how collaborative filtering matching operates on ratings matrices, and how description embeddings construct personalized content profiles. Part I establishes that foundation.

CHAPTER 1

Foundations — Explicit vs Implicit Feedback

Every recommendation system is only as good as the signals feeding it. Before a single model is trained, a senior engineer must decide what counts as evidence of preference, how strong that evidence is, and how to normalize the wild variance between a deliberate five-star review and an accidental tap.

A recommendation pipeline begins where the user leaves a trace. Those traces — ratings, reviews, clicks, dwell time, add-to-carts, purchases — are the raw fuel for every downstream stage: candidate generation, similarity scoring, ranking, and re-ranking. Get the feedback layer wrong and no amount of clever matrix factorization will save the system, because the model will faithfully learn a distorted picture of what people actually want.

This chapter defines the two families of signal, **explicit** and **implicit** feedback; explains how each is captured in a production schema; shows how heterogeneous signals are fused into a single numeric preference matrix; and dissects the reference project's feedback classifier so you can run it yourself and watch the numbers move. Throughout, the emphasis is on the engineering trade-offs a real system forces you to make, not textbook idealizations (Hu, Koren, & Volinsky, 2008).

1.1 Signal Taxonomy

Explicit vs. Implicit Feedback

Explicit feedback is an active, conscious statement of preference: a star rating, a written review, a thumbs-up, a wishlist add. It is high-fidelity — the user is literally telling you how they feel — but it is *sparse*, because most people never rate anything.

Implicit feedback is a passive behavioral trace: a page view, a click, a scroll, dwell time, a purchase. It is *abundant* — every session generates dozens of events — but it is *noisy* and *one-sided*: it tells you what a user engaged with, never explicitly what they disliked.

The single most important consequence of this split is the **missing-not-at-random** problem. In an explicit matrix, a blank cell means "not yet rated." In an implicit matrix, a blank cell is genuinely ambiguous: the user may dislike the item, may never have seen it, or may simply have run out of time. Treating implicit non-events as negative labels naively will teach the model that everything unseen is bad — a subtle but catastrophic bias.

1.1.1 Weighting and Normalization

Because signals arrive on incompatible scales, a production system maps each event type to a common numeric *preference* value. A rating is already on a 1–5 scale; a purchase is a strong positive; a view is weak. The reference project encodes exactly this mapping (note how it is reused verbatim across the collaborative-filtering, hybrid, and capstone modules):

$$w(\text{event}) = \begin{cases} v & \text{if type} = \texttt{rating} \quad (v \in [1,5]) \\ 5.0 & \text{if type} = \texttt{purchase} \\ 2.0 & \text{if type} = \texttt{click} \\ 1.0 & \text{if type} = \texttt{view} \end{cases} \tag{1.1}$$

Hu et al. (2008) formalize this as splitting each observation into a binary *preference* p_{ui} and a *confidence* c_{ui}: the more times a user views an item, the more confident we are the interaction is meaningful. The linear map above is the pragmatic, production-friendly approximation of that idea.

Table 1.1 contrasts the two streams across the dimensions that actually matter when you are designing schemas and loss functions.

Dimension	Explicit Feedback	Implicit Feedback
User effort	High (deliberate action)	Low (passive behavior)
Volume	Sparse ($< 5\%$ of users)	Abundant (every session)
Noise level	Low	High (mis-taps, bots, shared devices)
Scale	1–5 stars, binary	Continuous (dwell, counts)
Negative signal	Clear (low ratings)	Ambiguous (absence $\neq$ dislike)
Recency drift	Slow	Fast (intent shifts per session)
Cold-start value	High per data point	High in aggregate

Table 1.1: Contrasting feedback streams across the dimensions that drive schema and loss-function design.

⋆ Key Insight

Implicit feedback is the lifeblood of scale. Fewer than five percent of active users leave an explicit rating, so a system that relies on ratings alone starves. The winning production pattern is to *fuse* both: use abundant implicit signals to fight sparsity, and use scarce explicit signals to calibrate and de-bias the implicit ones.

1.2 Architecture: The Feedback Ingestion Layer

Before any recommender runs, feedback must be captured, typed, weighted, and persisted. Figure 1.1 shows the ingestion layer of the reference project: the Next.js lab UI emits interaction events, the FastAPI service normalizes them through the weight map, and two stores hold the

results — the `interactions` table for behavioral logs and the `memory_records` table for distilled explicit preferences with vector embeddings.

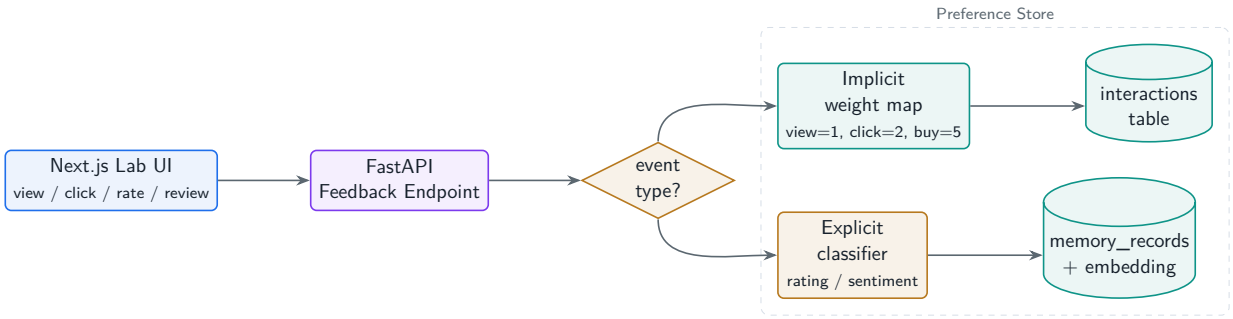

Figure 1.1: Feedback ingestion architecture: raw UI events are typed and weighted at the API boundary, then split into a behavioral interaction store and an embedded preference-memory store that downstream chapters consume.

1.3 Production Code: The Explicit Sentiment Classifier

Explicit textual feedback (a review) carries a polarity that a numeric star rating does not. The reference project ships a transparent, dependency-free lexicon classifier — deliberately simple so its behavior is fully auditable in an interview setting. It reads either an inline `text` argument or the user's most recent memory record, then scores polarity by counting matched sentiment tokens.

`app/course/commands.py: execute_classify_feedback`

```
async def execute_classify_feedback(
    session: AsyncSession,
    user_id: UUID,
    params: Dict[str, Any]
) -> CommandResponse:
    # Signal source: an inline review OR the user's latest stored memory.
    from_memory = str(params.get("from_memory", "false")).lower() in (
        "true", "1", "yes", "--from-memory")
    text = ""
    if from_memory:
        stmt = (select(MemoryRecord)
                .where(MemoryRecord.user_id == user_id)
                .order_by(MemoryRecord.timestamp.desc()))
        res = await session.execute(stmt)
        mem = res.scalars().first()
        if not mem:                          # graceful cold-start: ask to seed
            return CommandResponse(status="needs_seed",
                                   suggested_command="seed-memory-demo")
        text = mem.content
    else:
        text = params.get("text", params.get("query", "")).strip()

    # Auditable lexicon - the polarity dictionaries a reviewer can inspect.
    positive_words = {"good", "great", "love", "like", "awesome",
                      "perfect", "amazing", "excellent", "best"}
```

```
    negative_words = {"bad", "terrible", "hate", "dislike", "broken",
                      "worst", "poor", "waste", "useless"}
    words = [w.lower().strip(",.!?") for w in text.split()]
    pos_count = sum(1 for w in words if w in positive_words)
    neg_count = sum(1 for w in words if w in negative_words)

    # Map the token counts to a bounded [0,1] preference score.
    if pos_count > neg_count:
        sentiment = "POSITIVE"
        score = min(1.0, 0.5 + 0.1 * (pos_count - neg_count))
    elif neg_count > pos_count:
        sentiment = "NEGATIVE"
        score = max(0.0, 0.5 - 0.1 * (neg_count - pos_count))
    else:
        sentiment = "NEUTRAL"
        score = 0.5

    return CommandResponse(
        status="success",
        message=f"Sentiment: {sentiment} (Score: {score:.2f})",
        data={"sentiment": sentiment, "score": score})
```

1.3.1 How the code works, line by line

The function is an `async` SQLAlchemy handler that receives the authenticated `user_id` and a loosely-typed `params` dictionary parsed from the slash command. Three design decisions are worth calling out in an interview:

First, the **signal-source switch** (`from_memory`) lets the same endpoint score either an ad-hoc string or the user's own stored history — the latter is how explicit review text becomes a durable preference. Second, the **graceful cold-start** return (`status="needs_seed"`) never throws; it tells the UI exactly which seed command to run, which is the pattern every command in this project uses. Third, the score is **bounded and symmetric** around a 0.5 neutral midpoint, so it drops straight into the $[0, 1]$ preference space the hybrid and capstone rankers expect.

! Pitfall / Warning

A lexicon classifier is intentionally naive: it misreads negation ("not good"), sarcasm, and domain jargon. In production you would swap the token count for a fine-tuned transformer or an LLM call — but keep the same bounded-score contract so nothing downstream changes. Never let a model's output scale leak into the ranking layer.

1.4 Seed Data and Verifying in the UI

The reference project ships deterministic fixtures so every listing is reproducible. The memory-demo fixture (`app/db/fixtures/memory_demo.json`) seeds eight explicit-preference records across four users; Table 1.2 shows the two that drive this chapter.

user	type	content (explicit preference text)
alice	preference	User prefers high-quality wireless headphones with active noise cancellation and premium cordless electronics.
bob	preference	User is interested in computer algorithms, database architecture, and distributed system books.

Table 1.2: Two seeded explicit-preference records used to demonstrate the classifier and, later, content-based profiling.

Bring up the stack, seed the data, then exercise the classifier from the chat page:

Terminal: seed + run

```
# one-time: seed the demo preference memories
curl -XPOST localhost:8000/api/v1/course/seed/memory-demo -H "Authorization: Bearer
↪  $TOKEN"

# then, in the chat UI, type the slash command:
/classify-feedback text="This product is great and amazing!"
```

Output Response

```
Sentiment: POSITIVE (Score: 0.70)
```

Two positive tokens (`great`, `amazing`) and zero negatives give $0.5+0.1\times(2-0)=0.70$. Scoring the stored review instead (`/classify-feedback from_memory=true`) reads Alice's seeded preference and returns a positive score, demonstrating the memory path.

1.4.1 The frontend dispatch

The lab UI does not special-case course commands; it streams every slash command to the same chat endpoint. This is the real dispatcher from the Next.js page:

frontend: chat/threadId/page.tsx (command dispatch)

```
// Any string starting with "/" is dispatched to the backend command router.
const executeCommand = async (commandStr: string) => {
  const cleanCmd = commandStr.trim();
  setMessages((prev) => [...prev, { role: 'human', content: cleanCmd }]);
  const token = localStorage.getItem('access_token');
  const res = await fetch(`${BACKEND_URL}/chat/${threadId}`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json',
               Authorization: `Bearer ${token}` },
    body: JSON.stringify({ prompt: cleanCmd, model_name: 'llama3.1' }),
  });
  // ...stream the newline-delimited JSON chunks back into the transcript
};
```

1.5 Execution Sequence Diagram

Figure 1.2 traces one `/classify-feedback` invocation from keypress to rendered score, showing exactly which class touches the data and where the bounded score is produced.

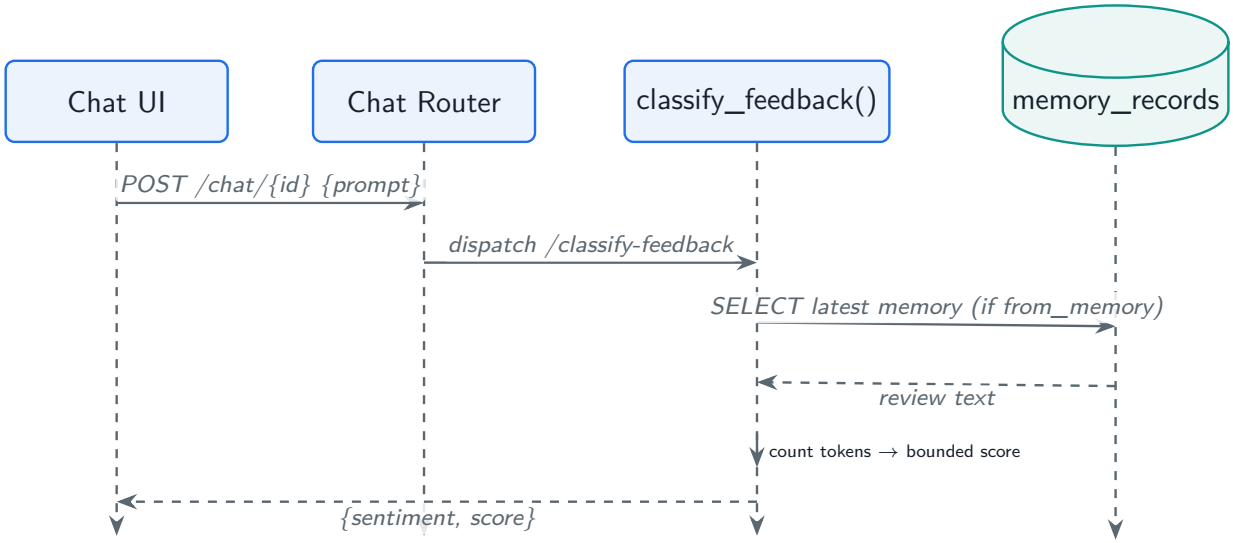

Figure 1.2: Execution flow of `/classify-feedback`: UI → chat router → command dispatcher → classifier, with an optional read of the memory store.

✓ Best Practice

Persist the raw event *and* the derived score. Storing only the derived 0.70 throws away the ability to re-weight later when your labeling policy changes. In this project the raw review lives in `memory_records`; the score is recomputed on demand.

1.6 Interview Questions & Answers

Q1.1 Why do implicit-feedback systems struggle to identify negative signals, and how do you handle it?

In explicit systems a one-star review is an unambiguous negative. In implicit systems, the *absence* of a click is ambiguous — it can mean dislike, non-exposure, or lack of time. This is the "missing-not-at-random" problem. Three production remedies: (1) **negative sampling** — treat a random subset of non-interacted items as weak negatives, which is what BPR does (Rendle et al., 2012); (2) **confidence weighting** — model every cell but weight observed interactions higher, per Hu et al. (2008), $c_{ui} = 1 + \alpha r_{ui}$; (3) **exposure logging** — record impressions so you can distinguish "seen but not clicked" from "never shown," turning a true negative into a usable label.

Q1.2 How would you model heterogeneous implicit signals (view, click, add-to-cart, purchase) in a single schema and a single preference matrix?

Store events in a narrow, append-only `interactions` table with columns `(user_id, item_id, type, value, timestamp, tenant_id)` — this is exactly the reference schema. Keep events raw; never pre-aggregate on write. At read time, collapse them into a preference value with an explicit weight map (view=1, click=2, purchase=5, rating=itself). This decouples the *logging* contract from the *modeling* policy: when you later decide a cart-add is worth 4.0, you change one dictionary and re-run, without a migration. For confidence-aware models, also carry the event *count* so you can compute $c_{ui} = 1 + \alpha \log(1 + \text{count})$.

Q1.3 The classifier scores "not good at all" as POSITIVE. Walk through the bug and how you would fix it in production.

The lexicon matches the token `good` and counts one positive, zero negatives, yielding 0.60 POSITIVE — negation is invisible to bag-of-words. Short term, add a negation window: if a negator (`not`, `never`, `no`) appears within two tokens before a polarity word, flip its sign. Medium term, replace the counter with a fine-tuned sentiment transformer or an LLM classifier behind the *same* bounded-score interface, so nothing downstream changes. The interview point is the abstraction boundary: the ranking layer must depend on a normalized $[0, 1]$ contract, not on the classifier's internals.

Q1.4 Explicit ratings suffer from selection bias. What is it, and what does it do to offline evaluation?

Users choose what to rate — they rate items they already like or strongly dislike, so the observed ratings are not a random sample of true preferences (missing-not-at-random). Naive RMSE on observed ratings then flatters the model, because it never scores the vast unrated region. Mitigations: inverse-propensity weighting (down-weight over-represented cells by their observation probability), collecting a small *randomized* exposure set for unbiased evaluation, and reporting ranking metrics (Recall@K, NDCG) on held-out interactions rather than rating RMSE alone. This is why Chapter 6 pairs RMSE with ranking metrics.

Q1.5 Design the feedback-ingestion layer for 50,000 events per second. What changes versus the synchronous handler shown here?

The synchronous "score-and-store" path shown here is fine for a lab but will not sustain 50k eps. Production shape: (1) the edge writes events to an append-only log (Kafka / Kinesis) — never a synchronous DB insert; (2) a stream processor applies the weight map and does light dedup/bot filtering; (3) events land in a columnar warehouse for training and a low-latency store (Redis / feature store) for serving; (4) heavy transforms (embeddings, sentiment) run async off the stream, not on the request path. Idempotency keys guard at-least-once delivery, and you separate the *write* scale (huge, cheap, async) from the *read* scale (small, hot, precomputed). The reference project keeps the read path identical so the concepts transfer directly.

From the Field

The most expensive mistake I see teams make is trusting a shiny model on top of an unlogged feedback layer. If you cannot answer "was this item even shown to the user?" your negatives

are fiction and your metrics are theater. Instrument exposure *first*; model second.

CHAPTER 2

Collaborative Filtering & Similarity Engines

Collaborative filtering is the idea that made recommenders famous: you do not need to understand an item to recommend it — you only need to find people who behaved like you and see what they liked. This chapter builds that engine from the sparse ratings matrix up, inside a strict multi-tenant boundary.

Content-based methods ask "what is this item like?" Collaborative filtering (CF) asks a cheaper and often more powerful question: "who is this user like?" By comparing rows of a user–item ratings matrix, CF surfaces items that similar users rated highly, with no knowledge of the items' contents at all (Sarwar, Karypis, Konstan, & Riedl, 2001). It is the backbone of Amazon's "customers who bought this" and Netflix's early prize-winning systems.

This chapter defines user–user CF, derives cosine similarity over sparse rating vectors, dissects the reference project's tenant-scoped similarity engine, and shows how a single `WHERE tenant_id = :tid` clause enforces the data isolation that a SaaS recommender lives or dies by.

2.1 The Ratings Matrix and Cosine Similarity

User–User Collaborative Filtering

Represent every user as a row vector over the item space, where entry r_{ui} is the (weighted) rating user u gave item i and unobserved cells are absent. Two users are *similar* if their observed vectors point in the same direction. To predict how u feels about an unseen item, take a similarity-weighted average of the ratings that u's neighbors gave it.

The matrix is overwhelmingly sparse — typically well over 99% empty — so we never materialize it densely. We compare only the items two users have *in common*. The direction-based similarity of choice is **cosine similarity**:

$$\text{sim}(u, v) = \cos(\theta) = \frac{\mathbf{r}_u \cdot \mathbf{r}_v}{\|\mathbf{r}_u\| \, \|\mathbf{r}_v\|} = \frac{\sum_{i \in I_u \cap I_v} r_{ui} \, r_{vi}}{\sqrt{\sum_{i \in I_u} r_{ui}^2} \, \sqrt{\sum_{i \in I_v} r_{vi}^2}} \tag{2.1}$$

Cosine is scale-invariant, which matters because a generous user who rates everything 4–5 and a harsh user who rates 1–3 can still be recognized as similar if the *shape* of their preferences agrees. Its main weakness — ignoring rating-mean differences — is addressed by **Pearson correlation** (mean-centered cosine); Table 2.1 compares the options.

Metric	Behavior	Use when
Cosine	Angle between vectors; scale-invariant	Implicit / positive-only data
Pearson	Cosine on mean-centered ratings	Explicit ratings with user bias
Jaccard	Overlap of interacted sets (ignores value)	Pure binary interactions
Adjusted cosine	Item-mean centered	Item-based CF variants

Table 2.1: Similarity metrics for collaborative filtering and when each is the right default.

⋆ Key Insight

Similarity is only meaningful over *shared* items. Two users with no items in common have similarity 0 regardless of how enthusiastic each is. This is why CF degrades in the cold-start and long-tail regimes and why we scope the whole computation to a tenant where overlap is dense.

2.2 Architecture: Tenant-Scoped CF Engine

In a multi-tenant SaaS recommender, ACME's ratings must never influence Beta Media's recommendations. Figure 2.1 shows how the reference project enforces this: the user's active tenant is resolved first, every interaction query is filtered by `tenant_id`, and the similarity engine only ever sees one tenant's slice of the matrix.

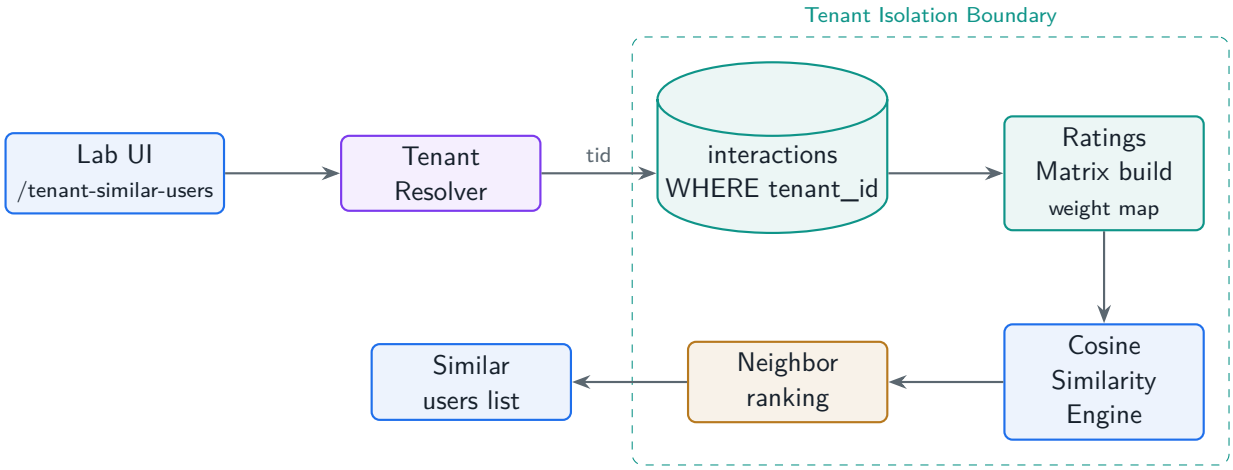

Figure 2.1: Tenant-scoped collaborative-filtering architecture. The tenant resolver gates every query, so the ratings matrix, similarity engine, and neighbor list are all isolated to a single workspace.

2.3 Production Code: The Similarity Engine

The core is a pure function over an in-memory ratings dictionary. It computes cosine similarity of the target user against every other user, intersecting on common items so sparsity is handled

naturally.

app/course/commands.py: compute_cosine_similarity

```
def compute_cosine_similarity(
    user_ratings: Dict[UUID, Dict[UUID, float]],
    target_user_id: UUID
) -> List[Dict[str, Any]]:
    if target_user_id not in user_ratings:
        return []
    target_vector = user_ratings[target_user_id]
    similarities = []
    for other_user_id, other_vector in user_ratings.items():
        if other_user_id == target_user_id:
            continue
        # Only items BOTH users rated contribute to the dot product.
        common_items = set(target_vector) & set(other_vector)
        if not common_items:
            similarities.append({"user_id": str(other_user_id), "score": 0.0})
            continue
        dot = sum(target_vector[i] * other_vector[i] for i in common_items)
        norm_t = math.sqrt(sum(v * v for v in target_vector.values()))
        norm_o = math.sqrt(sum(v * v for v in other_vector.values()))
        score = 0.0 if norm_t == 0 or norm_o == 0 else dot / (norm_t * norm_o)
        similarities.append({"user_id": str(other_user_id),
                             "score": round(score, 4)})
    similarities.sort(key=lambda x: x["score"], reverse=True)
    return similarities
```

The tenant-scoped handler wraps this with the isolation logic: resolve the tenant, pull only that tenant's interactions, collapse heterogeneous event types into ratings via the weight map, then rank neighbors.

app/course/commands.py: execute_tenant_similar_users (core)

```
active_tenant_id = mappings[0].tenant_id  # resolved from TenantUser
stmt_int = select(Interaction).where(Interaction.tenant_id == active_tenant_id)
interactions = (await session.execute(stmt_int)).scalars().all()

# Fuse view/click/purchase/rating into one preference scale (Chapter 1 map).
rating_map = {"rating": lambda v: v, "purchase": lambda v: 5.0,
              "view": lambda v: 1.0, "click": lambda v: 2.0}
user_ratings: Dict[UUID, Dict[UUID, float]] = {}
for inter in interactions:
    if not inter.item_id:
        continue
    val = rating_map.get(inter.type, lambda v: 1.0)(inter.value)
    user_ratings.setdefault(inter.user_id, {})[inter.item_id] = val

similarities = compute_cosine_similarity(user_ratings, user_id)
```

The SQL that enforces isolation is the quiet hero. The ORM emits exactly this, and the `tenant_id`

predicate is non-negotiable:

Tenant-isolated ratings pull

```
SELECT user_id, item_id, type, value
FROM   interactions
WHERE  tenant_id = :active_tenant_id;    -- hard isolation boundary
```

! Pitfall / Warning

The most dangerous CF bug in SaaS is a *missing* tenant predicate. Without it the query silently returns every tenant's data, the model trains on a leaked matrix, and one customer's behavior recommends products to another. Enforce the filter at the query layer and, ideally, again with Postgres row-level security.

2.4 Seed Data and Verifying in the UI

The tenant-demo fixture seeds two isolated workspaces. ACME Retailer contains Alice and Bob with overlapping electronics/book interactions; Table 2.2 shows the ACME rating slice after the weight map is applied.

user	item	type	weight
alice	SuperCordless Headphones	rating	5.0
alice	ProBook E-Reader	rating	4.0
bob	Intro to Algorithms	rating	5.0
bob	Designing Data-Intensive Apps	rating	4.5

Table 2.2: ACME Retailer ratings after weighting. Alice and Bob share the ACME tenant, so their vectors are eligible for comparison; Beta Media users are never in scope.

Terminal: seed + run

```
curl -XPOST localhost:8000/api/v1/course/seed/tenant-demo -H "Authorization: Bearer
↪  $TOKEN"
# then in the chat UI:
/tenant-similar-users
```

Output Response

```
Tenant-Isolated Collaborative Filtering (ACME Retailer):
- bob (ID: `...`): Cosine Similarity = 0.0000
```

The zero is instructive, not a bug: in the seed slice Alice rated only electronics and Bob only books, so their common-item set is empty and cosine is 0. Add one shared rating (or run `/generate-tenant-users`) and the score jumps — a live demonstration of the shared-item requirement from Section 2.1.

2.5 Execution Sequence Diagram

Figure 2.2 traces the request from the UI through tenant resolution, the isolated query, matrix assembly, and the similarity computation.

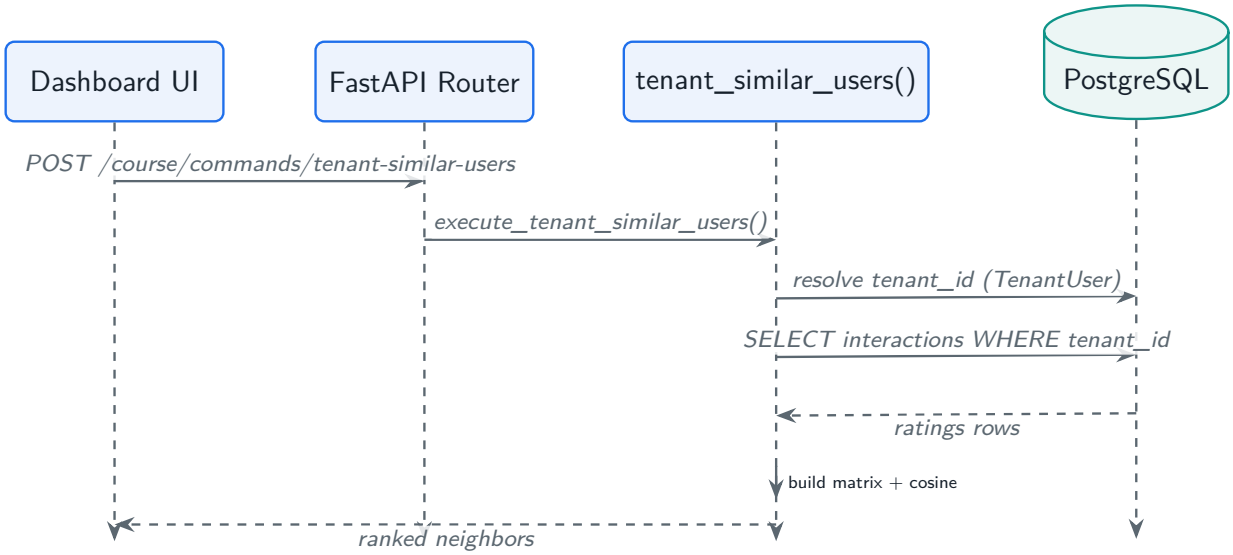

Figure 2.2: Execution flow of `/tenant-similar-users`: tenant resolution precedes every data access, guaranteeing the similarity matrix is built from one tenant only.

✓ Best Practice

Precompute and cache the top-k neighbor list per user offline (a nightly batch), and serve it from a key-value store. Real-time cosine over the full matrix is fine for a lab and small tenants, but production CF is almost always a precomputed neighbor table refreshed on a schedule.

2.6 Interview Questions & Answers

Q2.1 Contrast user–user and item–item collaborative filtering. Which does Amazon-scale production prefer, and why?

User–user CF finds neighbors of the *user* and recommends what they liked; item–item CF precomputes item-to-item similarities and recommends items similar to what the user already engaged with (Sarwar et al., 2001). Production at scale overwhelmingly prefers item–item, for three reasons: item similarities are far more *stable* over time than volatile user tastes, so they can be precomputed and cached; there are usually fewer items than users, making the similarity matrix smaller; and it degrades gracefully for new users (one click is enough to recommend neighbors of that item). User–user shines in dense, smaller communities where neighborhoods are rich — which is exactly the tenant-scoped regime in this project.

Q2.2 Why cosine over Euclidean distance for the ratings matrix, and when would you switch to Pearson?

Euclidean distance is dominated by vector *magnitude*: a user who has rated 100 items looks "far" from one who rated 5, even if their tastes align perfectly. Cosine measures the *angle*, so it is invariant to how many items or how generously a user rates — the property we want. Switch to Pearson (mean-centered cosine) when you have explicit ratings with strong per-user bias: one user's "3" is another's "5." Pearson subtracts each user's mean before the dot product, correcting for that bias. For implicit / positive-only data, plain cosine is the right default.

Q2.3 Two users in the same tenant return similarity 0. Walk through the possible causes.

Cosine is 0 exactly when the numerator is 0, i.e. no items in common (the seed-data case in this chapter), or when one user's norm is 0 (no rated items — a genuine cold-start user). It is *never* negative here because all weighted preferences are non-negative. The debugging path: check the intersection size first (`common_items`), then the norms. If overlap exists but the score still looks wrong, verify the weight map applied — a bug that leaves `view` as its raw `value` instead of 1.0 silently distorts the geometry.

Q2.4 The naive similarity computation is $O(n^2 m)$. How do you scale collaborative filtering to millions of users?

The pairwise approach is fine to hundreds of thousands of interactions but not to millions of users. Production paths: (1) **model-based CF** — factorize the matrix into low-rank user/item embeddings (Chapter 9) and reduce recommendation to a dot product; (2) **approximate neighbor search** — index user or item embeddings in an ANN structure like HNSW (Chapter 8) so neighbor lookup is sublinear; (3) **offline batch precompute** of top-k neighbors with locality-sensitive hashing to prune candidate pairs; (4) **minhash/LSH** on interacted-item sets for a cheap first-pass filter. The common thread: move the quadratic work offline and serve a precomputed table.

Q2.5 In a multi-tenant recommender, how do you guarantee one tenant's data never leaks into another's recommendations?

Defense in depth. At the query layer, every data access carries a `WHERE tenant_id = :tid` predicate resolved from the authenticated user's `TenantUser` mapping — as shown in this chapter. Below that, enable Postgres row-level security so even a forgotten predicate cannot return foreign rows. Above it, scope caches and precomputed neighbor tables by tenant key so a cache hit can never cross the boundary. Add an automated test that seeds two tenants and asserts zero cross-tenant neighbors. Isolation is a correctness *and* a compliance requirement; treat a leak as a Sev-1.

From the Field

Teams love to reach for deep models on day one. But on a fresh tenant with a few hundred interactions, a cached item–item cosine table beats a half-trained neural recommender every time — and it is debuggable at 2 a.m. Earn the right to add complexity by first exhausting the simple, explainable baseline.